Wholesale Stock Management SaaS — Scope Draft

Oct 27, 2026 · @Oporo.ke

What we're building

- Multi-tenant SaaS stock management for wholesalers (alcohol first; pharmacy and general later)
- Fast counter sales: customer says what they want, clerk keys it in, receipt prints on thermal printer
- Correct handling of nested packaging (box → six-pack → bottle; crate → 20/25 bottles)
- Fast product/price lookup, easy quotations
- Full stock management around it
- Backend in Axum (Postgres), frontend in Angular, built step by step as the A–Z Axum learning project

Core design decision: packaging

- Stock stored in **one base unit** per product (e.g. bottle). Never store "2 crates + 5 bottles"
- Each product has **pack levels**: bottle = 1, six-pack = 6, box = 4 six-packs = 24, crate = 25
- Each pack level has: name, quantity in base units, own selling price, own barcode
- Sale line: "3 crates" → system converts to base units for stock, prices at crate price
- Stock display converts back: "1,203 bottles = 48 crates + 3 bottles"
- Break-bulk: opening a box is just selling at a lower level; no manual "unpack" step needed (optional audit record)

Modules

MVP

- Tenants (flat subscription), branches (single shop or multi-branch), users, roles/permissions

- Products, categories, pack levels, barcodes per pack level
- Pricing: per pack level; optional price tiers (retail/wholesale/custom) — tenant setting chooses
- Fast POS screen: search by name/code/barcode scanner, keyboard-driven
- Offline selling at the counter, sync when internet returns
- Sales + receipts (thermal 58/80 mm), VAT-inclusive prices, excise shown where required
- Fiscal-integration ready: tax data stored per sale line, fiscal adapter interface in the backend (no TRA/KRA connection yet)
- Quotations → convert to sale
- Customers; credit sales as a tenant setting, with per-customer credit limits and balances
- Payments: cash, mobile money, bank, credit; split payments
- Stock: purchases/receiving (GRN), adjustments, stock counts, low-stock alerts
- Branch stock transfers (when tenant has more than one branch)
- Suppliers
- Reports: daily sales, stock on hand, stock movement, profit, cashier shifts

Alcohol-specific

- **Returnable empties (crates) — in MVP**: deposits charged and refunded, empties stock per customer
- Price changes by date (frequent supplier price changes)

Later

- Fiscal integration: TRA VFD/EFD (Tanzania), KRA eTIMS (Kenya), plugged into the fiscal adapter
- Pharmacy: batch numbers, expiry dates, FEFO, controlled-drug records
- Purchase orders
- iOS app

Thermal printing (needs decision)

- A browser web app cannot print raw ESC/POS directly to USB on Windows without workarounds

- **A. Small local print agent** (Rust, runs on the counter PC) — web app sends receipt to it; supports USB, Bluetooth and network printers. Most reliable.
- **B. Browser print dialog** (HTML receipt, printer driver installed) — simplest, slower, less control
- **C. Web Bluetooth / WebUSB** (Chrome only) — limited, fragile on Windows
- Superseded by offline-first: the native app shell prints directly (see Offline-first architecture)

Offline-first architecture

Every device works fully without internet and syncs when a connection returns.

- **Clients**: one Angular codebase on three targets — web (browser), desktop (Windows, Linux) and Android; desktop and Android via Tauri 2
- **Local database on each device** (SQLite): products, prices, customers, stock, sales, quotations
- **Server** (Axum + Postgres): source of truth across devices, branches and tenants; sync API
- **IDs**: generated on the device (UUID v7), so records never clash when merged
- **Stock as a ledger**: every change is an append-only movement (sale, receipt, adjustment, transfer, empties); stock on hand = sum of movements. Movements from different devices merge without conflicts
- **Sync**: device pushes its outbox of new records, then pulls changes since its last sync checkpoint
- **Master data** (products, prices, settings): edited online by managers; devices pull it. Offline edits limited by role
- **Receipt numbers**: per-device series (e.g. `B1-D2-000123`) so offline devices never duplicate
- **Negative stock**: possible when two offline devices sell the last units; allowed, flagged on sync for review
- **Printing**: native shell talks to the printer directly — USB on Windows, Bluetooth on Android

Platform decisions

- Native shell: **Tauri 2** — Windows, Linux, Android from one Rust shell

- Web: Angular in the browser, installable as a PWA, **full offline POS** (same features as the native apps)
- Local storage behind one Angular data interface: SQLite in Tauri apps; IndexedDB in the browser
- Web storage safety: request persistent storage, sync as soon as online, warn the user when unsynced records exist (browser data can be cleared)
- Printing: Tauri apps print directly (USB, Bluetooth); web uses the browser print dialog

Decisions (27 Oct 2026)

#	Question	Decision	Impact
1	Market	Tanzania and Kenya	Country per tenant; currency TZS/KES; fiscal integration later; system integration-ready
2	VAT/excise	Prices VAT-inclusive; excise on receipt	Tax breakdown stored per sale line
3	Returnable empties	Yes, crates	Empties module in MVP
4	Frontend	Angular	Angular SPA consumes the Axum API
5	Offline	Yes	Whole app offline-first; sync when online; Android app in scope
6	Barcode scanners	Supported	Barcode per pack level; scanner acts as keyboard input
7	Price tiers	Both; tenant chooses	Tenant setting: single price or tiered
8	Credit	Tenant setting	Credit on/off per tenant; limits per customer
9	Branches	Single shop and multi-branch	Branch model from day one; transfers when >1 branch
10	Billing	Flat	One plan price per tenant

Still open

- None. Scope locked; next is environment setup (step 0)

Build plan

Tick each step when it compiles, passes tests and works.

Phase 0 — Setup

- [ ] Step 0: Toolchain check (Rust, Node, Postgres/Docker) — every later step depends on matching versions
- [ ] Step 1: Monorepo layout (Cargo workspace: api, domain, sync types; Angular app; Tauri app) — shared types stay consistent across backend and clients
- [ ] Step 2: Local Postgres via Docker Compose — identical, reproducible database on every machine

Phase 1 — Backend foundation (Axum)

- [ ] Step 3: Minimal Axum server (tokio, Router, axum::serve, /health) — proves the stack runs
- [ ] Step 4: Typed configuration from env/.env — no hardcoded secrets; same binary for dev and prod
- [ ] Step 5: Tracing and logging (TraceLayer, request IDs) — needed to debug production issues
- [ ] Step 6: Error handling (AppError implementing IntoResponse) — one consistent JSON error format
- [ ] Step 7: Database pool and migrations (sqlx) — versioned schema, safe upgrades
- [ ] Step 8: App state and FromRef substates — shared resources reach handlers cleanly
- [ ] Step 9: Core middleware (CORS, compression, timeouts, body limit) and graceful shutdown — production-safe HTTP behaviour
- [ ] Step 10: Test harness (oneshot router tests, test database) — every later step ships with tests

Phase 2 — Tenancy and security

- [ ] Step 11: Tenants and branches — SaaS isolation; single shop or multi-branch
- [ ] Step 12: Users and password hashing (argon2) — secure accounts

- [ ] Step 13: Authentication (JWT access + refresh, device registration) — login that works for offline devices
- [ ] Step 14: Custom extractors (CurrentUser, TenantContext) and rejections — every handler knows who and which tenant
- [ ] Step 15: Roles and permissions (route_layer middleware) — cashier vs manager vs owner
- [ ] Step 16: Rate limiting and security headers — protects the public API

Phase 3 — Catalog

- [ ] Step 17: Categories and products CRUD (validation, pagination) — the base data everything uses
- [ ] Step 18: Pack levels and unit conversion — core rule: stock in base units, sold in any pack
- [ ] Step 19: Barcodes per pack level — scanner support
- [ ] Step 20: Pricing (per pack level, optional tiers, effective dates) — correct price on every line
- [ ] Step 21: Fast product search (Postgres trigram index) — instant lookup at the counter

Phase 4 — Stock ledger

- [ ] Step 22: Stock movement ledger and stock on hand — conflict-free stock across offline devices
- [ ] Step 23: Suppliers and goods receiving (GRN) — stock in
- [ ] Step 24: Adjustments and stock counts — fix shrinkage and errors with an audit trail
- [ ] Step 25: Branch transfers — move stock between branches
- [ ] Step 26: Returnable empties (crates) — deposits, refunds, empties per customer

Phase 5 — Sales

- [ ] Step 27: Customers and credit settings — credit on/off, limits, balances
- [ ] Step 28: Quotations — quote, then convert to sale
- [ ] Step 29: Sales (pack-unit lines, base-unit stock, tax per line, one DB transaction) — the core transaction
- [ ] Step 30: Payments (split, mobile money, credit) — every sale settles correctly
- [ ] Step 31: Per-device receipt numbering — no duplicate numbers offline

- [ ] Step 32: Fiscal adapter interface (no-op) — TRA/KRA plug in later without redesign
- [ ] Step 33: Cashier shifts (open, close, cash count) — cash accountability

Phase 6 — Sync

- [ ] Step 34: Sync protocol design (outbox, checkpoints, idempotency keys) — rules before code
- [ ] Step 35: Sync push endpoint — devices upload offline work
- [ ] Step 36: Sync pull endpoint — devices download changes since last sync
- [ ] Step 37: Conflict and negative-stock flags — review what offline merging produced
- [ ] Step 38: Live updates (WebSocket/SSE) — online devices pull changes immediately

Phase 7 — Reports and files

- [ ] Step 39: Report endpoints (sales, stock, movement, profit, shifts) — owner visibility
- [ ] Step 40: Exports (CSV/PDF) and bulk product import (multipart) — onboarding and accounting
- [ ] Step 41: OpenAPI docs (utoipa) — documented API for clients and integrations

Phase 8 — Angular frontend

- [ ] Step 42: Angular workspace, routing, layout — client foundation
- [ ] Step 43: Local data interface + IndexedDB implementation — offline storage in the browser
- [ ] Step 44: Client sync engine — push/pull, retry, unsynced warning
- [ ] Step 45: Auth screens — login, device registration
- [ ] Step 46: Product, pack and pricing screens — catalog management
- [ ] Step 47: POS screen (keyboard-driven, scanner) — fast counter sales
- [ ] Step 48: Receipt template and browser printing — web printing
- [ ] Step 49: Quotations, customers, stock, empties screens — remaining operations
- [ ] Step 50: Report screens — owner dashboards
- [ ] Step 51: PWA (service worker, persistent storage) — installable, offline web app

Phase 9 — Native apps (Tauri 2)

- [ ] Step 52: Tauri desktop shell (Windows, Linux) — native install
- [ ] Step 53: SQLite implementation of the data interface — durable offline storage

- [ ] Step 54: ESC/POS printing over USB (desktop) — direct thermal printing
- [ ] Step 55: Android build — mobile and tablet counters
- [ ] Step 56: Bluetooth printing (Android) — portable thermal printers

Phase 10 — SaaS and production

- [ ] Step 57: Tenant signup and flat subscription — sell the product
- [ ] Step 58: Backups and restore — no data loss
- [ ] Step 59: Deployment (Docker image, reverse proxy, TLS, VPS) — live server
- [ ] Step 60: CI/CD (tests, builds, app releases, auto-update) — safe, repeatable releases
- [ ] Step 61: Monitoring and alerts — know about failures first
- [ ] Step 62: Pilot with an alcohol wholesaler — real-world validation
- [ ] Step 63: Launch — open to customers